

# PropHunt: Automated Optimization of Quantum Syndrome Measurement Circuits

Joshua Viszlai  
University of Chicago  
Department of Computer Science  
Chicago, IL, USA  
viszlai@uchicago.edu

Satvik Maurya  
University of Wisconsin-Madison  
Department of Computer Sciences  
Madison, WI, USA  
smaurya@wisc.edu

Swamit Tannu  
University of Wisconsin-Madison  
Department of Computer Sciences  
Madison, WI, USA  
swamit@cs.wisc.edu

Margaret Martonosi  
Princeton University  
Department of Computer Science  
Princeton, NJ, USA  
mrm@cs.princeton.edu

Frederic T. Chong  
University of Chicago  
Department of Computer Science  
Chicago, IL, USA  
chong@cs.uchicago.edu

## Abstract

Fault-Tolerant Quantum Computing (FTQC) relies on Quantum Error Correction (QEC) codes to reach error rates necessary for large scale quantum applications. At a physical level, QEC codes perform parity checks on data qubits, producing syndrome information, through Syndrome Measurement (SM) circuits. These circuits define a code's logical error rate and must be run repeatedly throughout the entire program. The performance of SM circuits is therefore critical to the success of a FTQC system.

While ultimately implemented as physical circuits, SM circuits have challenges that are not addressed by existing circuit optimization tools. Importantly, inside SM circuits themselves errors are expected to occur, and how errors propagate through SM circuits directly impacts which errors are detectable and correctable, defining the code's logical error rate. This is not modeled in NISQ-era tools, which instead optimize for targets such as gate depth or gate count to mitigate the chance that any error occurs. This gap leaves key questions unanswered about the expected real-world effectiveness of QEC codes.

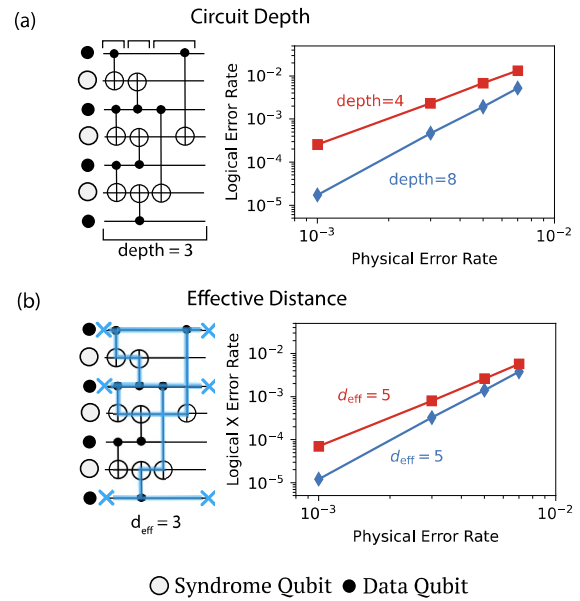
In this work we address this gap and present PropHunt, an automated tool for optimizing SM circuits for CSS codes. We evaluate PropHunt on a suite of relevant QEC codes and demonstrate PropHunt's ability to iteratively improve performance and recover existing hand-designed circuits automatically. We also propose a near-term QEC application, Hook-ZNE, which leverages PropHunt's fine-grained control over logical error rate to improve Zero-Noise Extrapolation (ZNE), a promising error mitigation strategy.

**CCS Concepts:** • Computer systems organization → Quantum computing; • Hardware → Quantum error correction and fault tolerance.



This work is licensed under a Creative Commons Attribution 4.0 International License.

## Imperfect Predictors of SM Circuit Performance



**Figure 1.** Common performance predictors targeted in Syndrome Measurement (SM) circuit design are imperfect. Data is from different SM circuits for a  $d = 5$  surface code. Red (square) plots indicate lower performing SM circuits despite having equal or better performance predictor values. Blue (diamond) plots indicate higher performing SM circuits despite having equal or worse performance predictor values.

**Keywords:** Quantum Computing; Quantum Error Correction; Syndrome Measurement Circuit; Error Mitigation

## 1 Introduction

Quantum Computing presents transformative potential to key applications in chemistry, material science, and cryptography. However, these applications require program sizes beyond the reach of noisy, physical quantum circuits. To

address this, researchers are focused on delivering Fault-Tolerant Quantum Computing (FTQC), where programs rely on an abstraction of Quantum Error Correction (QEC) to repeatedly detect and correct errors during execution.

QEC codes are the building blocks of FTQC, and recent years have seen an explosion in QEC code designs [7, 17, 25, 29, 33] and proposed implementations [7, 37, 40, 56, 58, 64]. While theoretical evaluations can inform promising QEC codes, full evaluations require a compilation to Syndrome Measurement (SM) circuits which implement a code’s parity checks as a physical circuit, enabling the detection of errors. Despite being key to QEC performance, techniques and tools for creating SM circuits in general are quite limited. Part of this stems from the design space complexity of SM circuits. For moderate code sizes, these involve hundreds of qubits and gates. For example, the well known surface code with a code distance of  $d = 11$  can be implemented with a SM circuit using 241 qubits and 440 CNOT gates. Additionally, SM circuits have specific constraints on CNOT ordering to preserve stabilizer commutation of the target QEC code. This is not modeled in general-purpose, physical circuit optimization tools [26, 35, 69]. Both of these features place SM circuit design beyond the reach of many existing approaches to circuit optimization.

Prior work has tamed the complexity of SM circuits in one of three ways. ① hand-designed circuits tailored to specific codes [28, 55, 56]. ② brute-force searches over a reduced design space [4, 7, 23]. And ③ circuit synthesis targeting imperfect performance predictors [49, 66]. However, none of these approaches are sufficient to address the growing space of QEC codes. Relying on hand-designed circuits is intensive, and will not always be tractable. For example, codes with random structure present little opportunity for hand-design. Relying on brute-force searches is also unscalable. While feasible for small codes, it quickly becomes intractable for the moderate or large code sizes necessary for FTQC. Finally, we present counter examples, shown in Figure 1, to common performance predictors used in designing SM circuits: circuit depth and effective code distance. While circuit depth is important for coherence-limited systems, Figure 1(a) shows depth alone is insufficient to capture the resulting logical error rate. A minimum depth circuit that has poorly ordered CNOT gates creates problematic error propagation through a SM circuit. This can then be outperformed by a circuit with well ordered CNOT gates, even if it is not minimum depth. Figure 1(b) shows just using the effective code distance: the minimum number of errors needed to create an undetected logical error, is also insufficient. Performance gaps can exist between SM circuits with the same effective code distance.

Addressing these shortcomings can significantly improve resulting SM circuit quality. For example, existing tools that target circuit depth without considering how errors propagate through CNOTs [66] can suffer a 10x increase in logical error rate for surface codes based on the data in Figure 1.

Our approach to SM circuit optimization has key differences compared to prior work. ① We optimize SM circuits at the granularity of individual CNOTs, exploring a fuller design space. ② We efficiently search through the larger design space by identifying and minimizing *ambiguity* in the SM circuit. Based on algorithmic insights, we introduce a generally applicable tool, PropHunt, for automated optimization of SM circuits. Applying our tool to surface codes, we show PropHunt is able to automatically recover the performance of hand-designed circuits. Furthermore, we find applying PropHunt to specific Lifted Product (LP) codes and Random Quantum Tanner (RQT) codes successfully optimizes SM circuit performance, reducing logical error rates by 2.5x-4x compared to a standard coloration circuit [56] at a physical error rate of 0.1%. We additionally motivate a near-term QEC application of PropHunt. We show intermediate SM circuits created by PropHunt can serve as useful, low-overhead error mitigation gadgets with Zero-Noise Extrapolation (ZNE) which we term Hook-ZNE. Our evaluations show Hook-ZNE improves over existing QEC+ZNE solutions, reducing error by 3x-6x compared to Distance-Scaling ZNE [60].

We summarize our contributions as follows:

- A novel solution for finding min-weight logical errors in a circuit-level decoding graph, allowing for efficient, parallelized circuit optimization.
- An approach for optimizing SM circuits based on modifying error propagation to minimize ambiguity, directly addressing logical error rate rather than using imperfect performance predictors.
- PropHunt, a tool for optimizing SM circuits based on our approach that minimizes ambiguity.
- SM circuits for select Lifted Product and Random Quantum Tanner codes that have 2.5x-4x lower logical error rates compared to known SM circuits.
- An additional application of PropHunt, Hook-ZNE, that uses intermediate SM circuits to improve Zero-Noise Extrapolation for near-term FTQC circuits.

## 2 Background

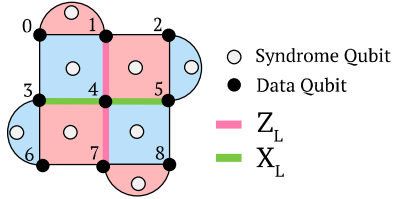
For a broader introduction to quantum computing, see [14, 36, 45]. Here, we focus on the essentials of quantum error correction (QEC) relevant to this work—stabilizer codes, their measurement circuits, and the role of gate ordering in preserving error resilience.

### 2.1 Quantum Stabilizer Codes

An  $[[n, k, d]]$  QEC code encodes  $k$  logical qubits into  $n$  physical qubits. The *code distance*  $d$  is the minimum number of physical qubit errors that can cause an undetectable logical error, equivalently the weight of the smallest logical operator.

The surface code is a widely studied stabilizer code defined on a two-dimensional qubit lattice. Data qubits store the encoded quantum information, while *syndrome qubits* measure multi-qubit parity checks, known as *stabilizers*, which detect the presence of errors without collapsing the logical state. A rotated surface code of distance  $d$  contains  $d^2$  data qubits and  $d^2 - 1$  syndrome qubits, evenly divided between  $X$ -type stabilizers (which detect  $Z$  errors) and  $Z$ -type stabilizers (which detect  $X$  errors).

## 2.2 Example: $d = 3$ Rotated Surface Code



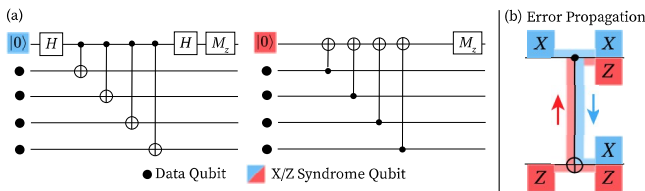
**Figure 2.** A  $d = 3$  surface code. Blue faces are  $X$ -type stabilizers and red faces are  $Z$ -type stabilizers.

Consider the  $d = 3$  rotated surface code, shown in Figure 2. The code has nine data qubits arranged in a  $3 \times 3$  grid and eight syndrome qubits implementing four  $X$ -type and four  $Z$ -type stabilizers. These can be expressed as parity-check matrices  $H_X$  and  $H_Z$ :

$$H_X = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}, \quad H_Z = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

Each row corresponds to a stabilizer, and a 1 in column  $j$  indicates that the stabilizer acts on data qubit  $j$ .

## 2.3 From Check Matrices to Ideal SM Circuits



**Figure 3.** (a) Example syndrome measurement circuits for measuring an  $X$  check (left) and a  $Z$  check (right). (b) CNOT error propagation rules.

The matrices  $H_X$  and  $H_Z$  directly specify a syndrome measurement (SM) circuit. As shown in Figure 3a, for an  $X$ -type stabilizer (row of  $H_X$ ), the corresponding circuit prepares an ancilla in the  $|+\rangle$  state, applies a CNOT from the ancilla (control) to each involved data qubit (target), and measures the ancilla in the  $X$  basis. For a  $Z$ -type stabilizer (row of

$H_Z$ ), the ancilla is prepared in  $|0\rangle$ , and for each involved data qubit, a CNOT is applied with the data qubit as control and the ancilla as target, followed by a  $Z$ -basis measurement of the ancilla.

Running all stabilizer circuits produces the syndrome vectors

$$s_X = H_X e_Z \pmod{2}, \quad s_Z = H_Z e_X \pmod{2},$$

where  $e_Z$  and  $e_X$  are binary vectors indicating  $Z$  and  $X$  errors on the data qubits, respectively.

## 2.4 From Ideal SM Circuits to Logical Errors

A *decoder* uses the syndrome vectors  $s_X$  and  $s_Z$  to try and deduce the errors  $e_Z$  and  $e_X$ . It can then infer the state of the logical qubits through the logical observable matrices  $L_X$  and  $L_Z$ . For the  $d = 3$  rotated surface code these are:

$$L_X = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}, \quad L_Z = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Then computing

$$l_Z = L_X e_Z \pmod{2}, \quad l_X = L_Z e_X \pmod{2},$$

results in the logical error vectors  $l_Z$  and  $l_X$ , indicating if a logically encoded qubit experienced a  $Z$  or  $X$  error, respectively.

## 2.5 Syndrome Generation and Decoding Examples

We illustrate the process of syndrome generation and decoding for  $X$  errors, which are detected by  $H_Z$ .

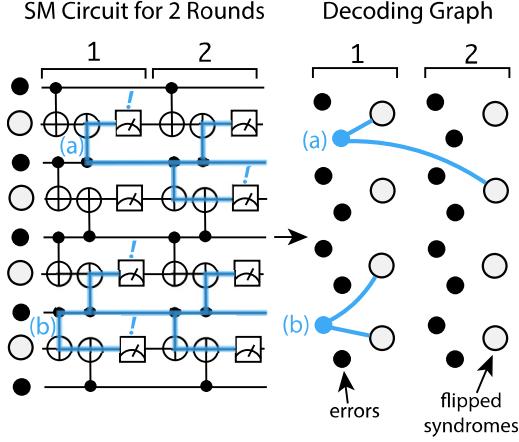
**Correctable Error.** A single  $X$  error on data qubit 5 corresponds to  $e_X = (0, 0, 0, 0, 1, 0, 0, 0, 0)^T$ . Since qubit 5 participates in rows 1 and 2 of  $H_Z$ , the resulting syndrome is  $s_Z = (1, 1, 0, 0)^T$ , which uniquely identifies qubit 5 as faulty.

The decoder determines the logical error is  $l_X = (1)^T$ , indicating a logical  $X$  error occurred. The decoder can then flip the logical qubit back, fully restoring the code state.

**Uncorrectable Error.** Consider  $X$  errors on qubits  $\{1, 5, 9\}$ :  $e_X = (1, 0, 0, 0, 1, 0, 0, 0, 1)^T$ . Here,  $s_Z = (0, 0, 0, 0)^T$ , indicating the pattern is undetected by all stabilizers. However,  $l_X = (1)^T$ , indicating the pattern results in a logical  $X$  error. Since an undetected logical error was created, the code state cannot be restored.

## 2.6 Noisy SM Circuits

In practice, the CNOT gates in SM circuits are imperfect: faults may occur on either the data or ancilla qubits at any point during the circuit. The fundamental property of QEC is that as long as the number and arrangement of such faults remain below the code's threshold, the decoder can still recover the logical state.



**Figure 4.** A circuit-level model for a noisy SM circuit. Decoding graph edges are only drawn for example errors (a) and (b).

Error propagation through a CNOT (Figure 3b) follows deterministic rules:

$$\begin{aligned} X_c &\rightarrow X_c X_t, & Z_c &\rightarrow Z_c, \\ X_t &\rightarrow X_t, & Z_t &\rightarrow Z_c Z_t, \end{aligned}$$

where  $c$  denotes the control and  $t$  the target qubit. These rules explain how data errors can flip stabilizer outcomes (desirable for detection), but also how ancilla errors, called hook errors, can spread to multiple data qubits (potentially harmful).

### 2.7 The Circuit-Level Model

To capture the behavior of noisy SM circuits, a *circuit-level* model is used. As gate errors can often be modeled as random Pauli gates, a circuit-level model statically *propagates* errors through a SM circuit to deduce the set of syndromes each error will flip. As a result, in the circuit-level model the code-level check and logical observable matrices are replaced by circuit-level counterparts,  $H$  and  $L$ , that replace qubits with gate errors.  $H$  maps gate errors (columns) to the set of syndromes they flip (rows), and  $L$  maps the same gate errors (columns) to the set of logical observables they flip (rows).

As shown in Figure 4, the ordering of CNOTs influences when errors are first detected. For example, an  $XX$  CNOT error, (a), is detected by one syndrome qubit in the first round and a different syndrome qubit in the second round. Due to different error propagation, another  $XX$  CNOT error, (b), is detected by two syndrome qubits in the first round.

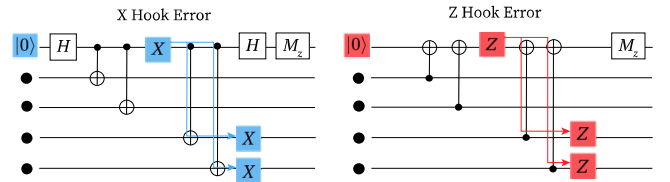
Importantly, since different SM circuits propagate errors differently, they correspond to different circuit-level check matrices,  $H$  and  $L$ .

Circuit-level matrices often include errors from  $d$  rounds of an SM circuit, and as a result can be significantly larger than the original stabilizer and logical observable matrices. For example, stabilizer matrices  $H_Z$  and  $H_X$  each have 49

columns for a  $d = 7$  surface code, but the circuit-level check matrix  $H$  can have  $> 15,000$  columns corresponding to possible errors.

### 2.8 Hook Errors

A *hook error* occurs when an ancilla fault in an SM circuit propagates to multiple data qubits via the CNOT error propagation rules. Consider a  $Z$ -type check on qubits  $\{q_1, q_2, q_3, q_4\}$ , with the ancilla as the target of all CNOTs. A  $Z$  error on the ancilla after the second CNOT will propagate back to the control data qubits of subsequent CNOTs, creating a correlated multi-qubit fault. If these qubits overlap with a minimum-weight logical operator, the fault can reduce the *effective code distance* and cause a logical error with fewer than  $d$  faults.



**Figure 5.** Worst case X and Z hook errors for weight 4 checks.

We can view a hook error on a weight  $w$  check as applying part of the corresponding stabilizer operator and so a hook error spread to  $w' > \lfloor \frac{w}{2} \rfloor$  data qubits can instead be interpreted as an error on  $w - w'$  data qubits by multiplying by the stabilizer operator. As a result in the worst case a hook error can cause a single error to become a multi-qubit error on  $\lfloor \frac{w}{2} \rfloor$  data qubits, as shown in Figure 5.

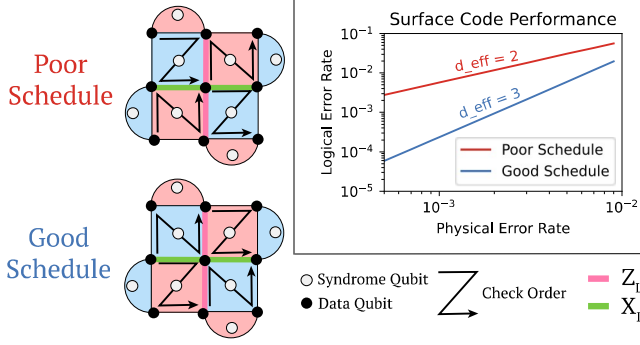
### 2.9 Effective Code Distance

The effective code distance  $d_{\text{eff}}$  is the minimum number of physical faults that create an undetected logical error for a given SM circuit. Poor CNOT ordering can reduce  $d_{\text{eff}}$  by allowing early ancilla faults to overlap with logical operators. Carefully chosen gate schedules can prevent such overlaps, ensuring  $d_{\text{eff}} = d$ .

While Figure 1 shows  $d_{\text{eff}}$  is not a perfect predictor of performance by itself, it is still meaningful. An SM circuit with a lower  $d_{\text{eff}}$  can be expected to perform worse, and determining  $d_{\text{eff}}$  is useful in explaining SM circuit performance.

## 3 Motivation

Currently there is a lack of approaches to optimize SM circuits. This can leave designers in the dark about the actual effectiveness of their QEC code. NISQ-era tools for optimizing circuits focus on minimizing gate counts and circuit depth to reduce the chance of physical errors occurring. For SM circuits, however, errors are *expected to occur* and must be tolerated. This presents a novel challenge as a SM circuit dictates how errors *propagate* into syndrome patterns that



**Figure 6.** Comparing the logical performance of a  $d = 3$  surface code when using two different CNOT schedules. The check order indicates the order a syndrome qubit performs CNOTs with its connected data qubits.

are then decoded. However, generally-applicable, automated solutions for SM circuit design are limited. Instead research on a QEC code where a circuit-level analysis is present will often have a section dedicated to how SM circuits are designed. Such solutions are generally hand-tailored to the code of interest and not applicable to other codes. Furthermore, as we point out in Figure 1, common targets used in SM circuit design are imperfect predictors of circuit performance. These features not only inhibit research progress, as designing SM circuits is a non-trivial task that must be redone for each code, but also inhibits code comparisons. Current attempts to compare codes at a circuit-level can struggle to disentangle whether performance differences are caused by a chosen SM circuit or by a code itself.

Despite the current lack of tooling, it’s well known that SM circuits can have a significant influence on a code’s logical error rate. Here, we present a motivating example of this and discuss how prior work has found performant SM circuits.

### 3.1 Motivating Example: Surface Code

The surface code has a well known schedule of CNOTs for a good SM circuit often referred to as the ‘N-Z’ schedule [55]. This schedule relies on the observation that any logical X (Z) operator in the surface code is horizontal (vertical). The ‘N-Z’ schedule therefore orders the CNOTs between check qubits and data qubits such that the worst case X (Z) hook errors are perpendicular to the X (Z) logical operator.

Figure 6 compares two SM circuits for a  $d = 3$  surface code: one that does not use the proper ‘N-Z’ schedule and one that does. The check order indicates the order a syndrome qubit performs CNOTs with its connected data qubits. We can see that using the proper ‘N-Z’ schedule has a notable impact on the code’s resulting logical error rate and that using a poor schedule can reduce  $d_{\text{eff}}$ . However, the ‘N-Z’ schedule is a hand-designed schedule for a SM circuit. In general, we would like an automated way to ① recover the

‘N-Z’ schedule for the surface code, and ② find similarly performant schedules for other QEC codes.

Interestingly, the sensitivity of logical error rate to different SM circuits is code-specific. As an example, for hypergraph-product codes it’s known that all SM circuits have  $d_{\text{eff}} = d$  [34]. Alternatively for the Steane code, we can note that all CNOT orderings produce hook errors that are distance-reducing, necessitating alternative approaches to SM circuit design. For most QEC codes, however, variance in logical error rate for possible SM circuits is unknown, motivating an automated approach for finding SM circuits.

## 4 Error Ambiguity Explained

Our approach views logical errors as stemming from *ambiguity*. Here we provide a formal definition of ambiguity and a worked out example for the surface code.

While the effective code distance  $d_{\text{eff}}$  describes a minimum weight logical operator at the circuit-level, the logical error rate scales as  $O(p^{\lceil d_{\text{eff}}/2 \rceil})$ . Logical faults occur due to the decoder *mispredicting* the state of the logical qubits as a result of *ambiguous errors*. Formally, we can define ambiguous errors as two sets of errors,  $e_1$  and  $e_2$ , such that  $He_1 = He_2$  and  $Le_1 \neq Le_2$ . In other words, two error patterns that have the same syndromes but different logical effects. A most likely error (MLE) decoder will therefore predict the most likely of the two to have occurred. As a result, mispredictions will occur with a probability proportional to the least likely of the two errors. We can further note that the union,  $e_L = e_1 + e_2 \pmod{2}$ , is an undetected logical error since  $He_L = He_1 + He_2 = 0 \pmod{2}$ , and  $Le_L = Le_1 + Le_2 \neq 0 \pmod{2}$ . Therefore a logical error of weight  $d_{\text{eff}}$  corresponds to many different sets of ambiguous errors, and closely balanced ambiguous errors,  $e_1$  with weight  $w_1 = \lfloor d_{\text{eff}}/2 \rfloor$  and  $e_2$  with weight  $w_2 = \lceil d_{\text{eff}}/2 \rceil$ , are the most likely failure cases.

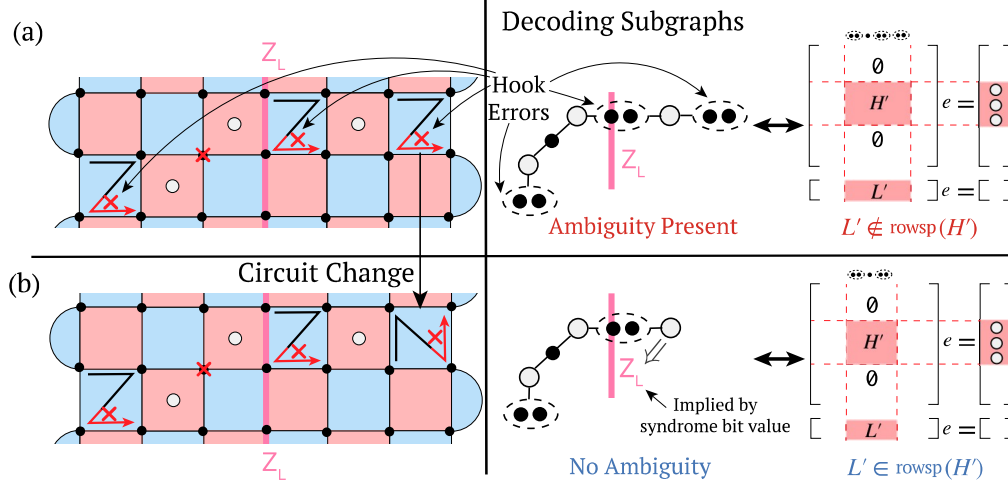
### 4.1 Checking Ambiguity

As discussed in [62], the presence of ambiguous errors can be detected efficiently. Given a subset of syndromes  $s'$  there can be an ambiguous explanation if  $L' \notin \text{rowsp}(H')$ , where  $H'$  and  $L'$  are sub-matrices of  $H$  and  $L$  corresponding to errors connected only to the syndromes  $s'$ .

A key insight is since  $H$  and  $L$  change with the SM circuit, as discussed in Section 2.7, we can check if for some modified SM circuit  $L' \in \text{rowsp}(H')$ , allowing  $s'$  to be decoded unambiguously and removing a logical fault pathway.

### 4.2 Comparing Ambiguity in SM Circuits

Figure 7a shows a logical error in a  $d = 7$  surface code corresponding to a reduced code distance of 4 due to hook errors from poorly scheduled CNOTs. Looking at the subgraph of the circuit-level decoding graph corresponding to the errors



**Figure 7.** A  $d = 7$  surface code with one CNOT ordering (a) and a modified CNOT ordering (b) from the perspective of ambiguity. A logical error in (a) corresponds to a decoding subgraph containing ambiguity. Any assignment of syndrome bit values that is decodable on the subgraph can be decoded in two ways that each result in a different effect on  $Z_L$ . The same decoding subgraph in (b) no longer has ambiguity as the state of  $Z_L$  is implied by the right-most syndrome bit. We can further verify a logical error has been removed by checking that the updated error mechanisms don't correspond to a new logical error on different syndrome bits.

involved, we can see the logical error exists due to the possibility of ambiguity. Any assignment of syndrome bit values that is decodable on the subgraph can be decoded in two ways that each result in a different effect on  $Z_L$ . For example, the middle syndrome being flipped can be explained by either the two hook errors on the right or the single error and hook error combination on the left. As both are equally likely, the decoder will not be able to reliably determine which case occurred. And since the first case flips the  $Z$  observable and the second does not, this will lead to a logical error.

In Figure 7b we can see that for a modified CNOT ordering, the same decoding subgraph no longer has the possibility of ambiguous errors. The effect on  $Z_L$  is implied by the right-most syndrome bit for any assignment of syndrome bit values decodable on the subgraph. We can further verify we've removed a potential logical error by checking that the updated errors, the three hook errors and single error, are also no longer a logical error.

## 5 PropHunt

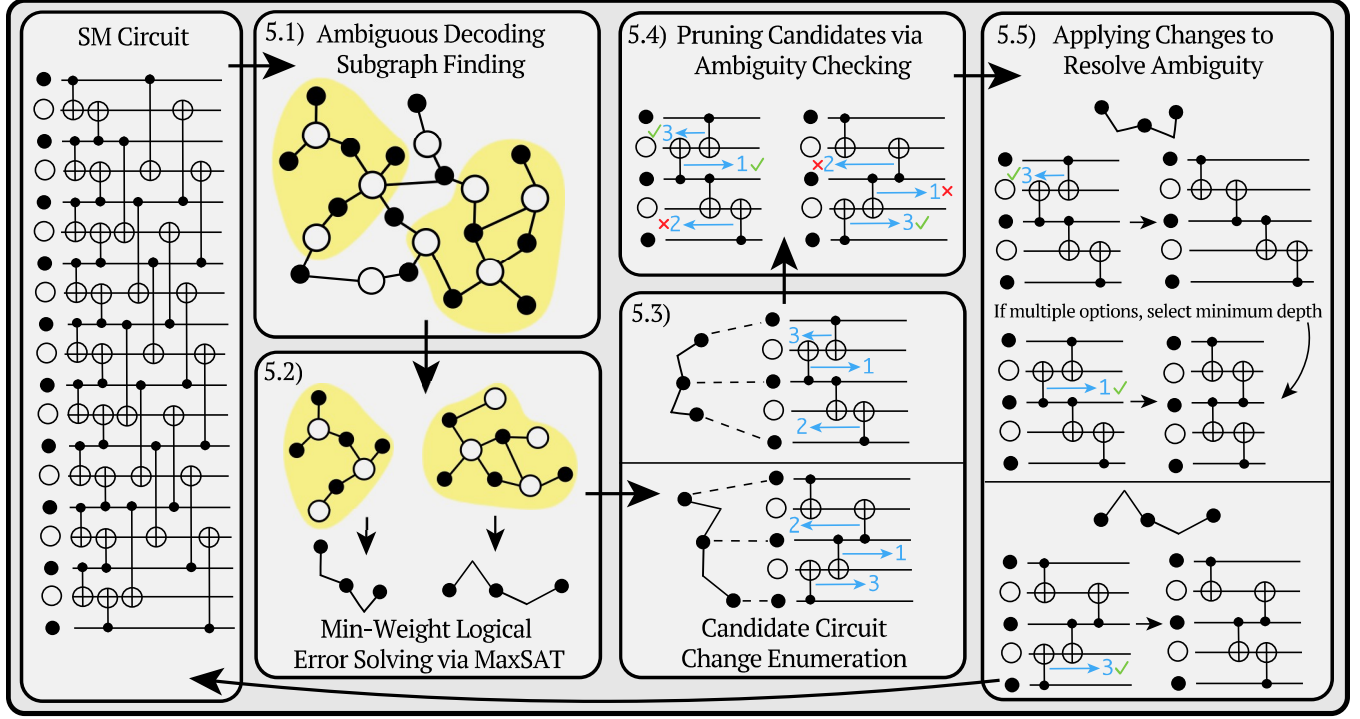
In this section we introduce a novel approach for automated optimization of SM circuits. Figure 8 gives an overview of our approach. First, a circuit-level decoding graph is produced from the SM circuit. Second, random subgraphs are expanded until  $L' \notin \text{rowsp}(H')$  indicating the presence of ambiguous errors. Third, min-weight logical errors are found in each subgraph using a MaxSAT formulation. Fourth, SM subcircuits corresponding to each logical error are modified to change the error propagation and remove the ambiguity.

An end-to-end example of this idea can be understood using the surface code. Starting from the poor schedule shown in Figure 2, our goal is to iteratively identify subgraphs shown in Figure 7 and apply circuit changes that resolve the ambiguity, such as the one in Figure 7b. Done repeatedly, a SM circuit of comparable performance to the good schedule shown in Figure 2 should be produced.

### 5.1 Ambiguous Decoding Subgraph Finding

We operate on bipartite circuit-level decoding graphs with sets of nodes corresponding to syndromes and errors. An edge between a syndrome node and an error node indicates that error flips that syndrome. To find possible ambiguous errors, we begin by selecting random error nodes as starting points. We additionally add all syndrome nodes connected to the error node to the subgraph. At each step we expand the subgraph by adding an additional error node and its connected syndrome nodes. Furthermore, any error nodes connected only to syndrome nodes already in the subgraph are automatically included. Subgraphs correspond to submatrices  $H'$  and  $L'$  and so after each step we check if  $L' \notin \text{rowsp}(H')$ , indicating whether the subgraph contains an ambiguous error. If an ambiguous error is present in a subgraph we halt its expansion.

We note that subgraphs containing disconnected components cannot contain a single ambiguous error spanning more than one connected component. During expansion, we therefore restrict ourselves to connected subgraphs—only error nodes adjacent to an already connected syndrome node are considered.



**Figure 8.** An overview of PropHunt’s iterative optimization loop.

## 5.2 Solving for Minimum Weight Errors

MaxSAT is a useful framework in solving for minimum weight logical errors in QEC. Here we describe how PropHunt uses MaxSAT to solve for min-weight logical errors in ambiguous subgraphs. Our formulation is similar to the approach in Stim [16] for finding minimum weight logical errors.

For a subgraph with error nodes  $E'$ , syndrome nodes  $S'$ , and submatrices  $H'$ ,  $L'$ , we first define variables  $E'_i$  and  $S'_i$  for all nodes in the subgraph. We then define

$$S'_i = E'_j \otimes \dots \otimes E'_k \text{ where } H_{i,j} = 1, \dots, H_{i,k} = 1$$

as a set of hard constraints enforcing syndromes as the parity of their connected errors. Similarly, we can define logical observable variables  $L'_i$  and a set of hard constraints

$$L'_i = E'_j \otimes \dots \otimes E'_k \text{ where } L_{i,j} = 1, \dots, L_{i,k} = 1$$

enforcing the logical observables as the parity of their connected errors.

Naively converting multivariate XOR clauses into Conjunctive Normal Form (CNF) is known to hinder solver performance as the resulting clause is exponential in size [41]. To avoid this, we introduce auxiliary variables ( $a_i$ ) in a standard, tree-like structure:

$$\begin{array}{c} S'_i = a_1 \otimes a_2 \\ \swarrow \quad \searrow \\ a_1 = E'_1 \otimes E'_2 \quad a_2 = E'_3 \otimes E'_4 \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ E'_1 \quad E'_2 \quad E'_3 \quad E'_4 \end{array}$$

We then add additional hard constraints enforcing that the assigned error pattern is undetected by all stabilizers

$$\bigvee_i S'_i = \text{False}$$

and flips at least one logical observable

$$\bigvee_i L'_i = \text{True}$$

Finally we add soft constraints for each error  $E_i$

$$E_i = \text{False}$$

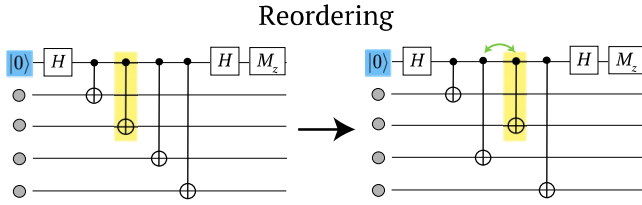
so the solver optimizes for the fewest error assignments while satisfying the hard constraints. This produces an optimal solution corresponding to a minimum weight logical error in the subgraph.

To solve, we use Z3 [12] to simplify the model and to apply the Tseitin transformation [57] to convert to CNF. We then use the Loandra MaxSAT solver [2] with a timeout of 360 seconds run on a Intel Xeon Silver 4116 CPU core.

### 5.3 Candidate SM Circuit Change Enumeration

As described in Section 2.7, in a circuit-level noise model errors occur during gates in the SM circuit. We can therefore map an error back to the gate that caused it. Using this, for each found minimum-weight logical error we enumerate a set of candidate SM circuit changes based on the involved gate errors. We implement two types of circuit changes, both of which modify the propagation order and the corresponding set of syndromes flipped by the circuit-level error.

**5.3.1 Reordering Changes.** We define *reordering* as changing the order of data qubits a syndrome qubit interacts with. This changes the set of data qubits involved in hook errors which then corresponds to a different set of flipped syndrome bits.



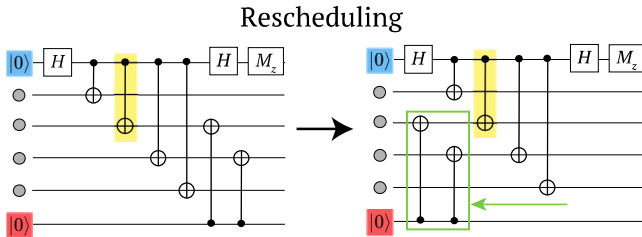
**Figure 9.** A reordering change to modify the behavior of a circuit-level error corresponding to the highlighted CNOT.

If the circuit-level error being modified is a hook error we enumerate  $w - 1$  reordering changes

$$\{q_1 \dots q_i \dots q_w\} \rightarrow \{q_1 \dots q_j, q_i \dots q_w\} \forall j \in \{1 \dots w\} \setminus \{i\}$$

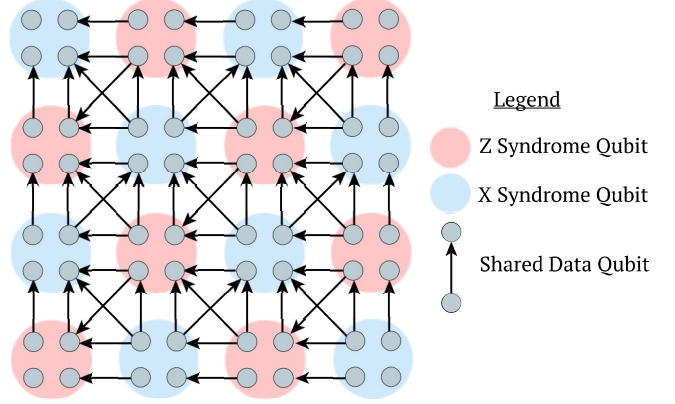
where  $w$  is the stabilizer weight and  $q_i$  is the data qubit involved in the CNOT that caused the hook error. Each of these changes corresponds to reordering another data qubit,  $q_j$ , before  $q_i$ .

**5.3.2 Rescheduling Changes.** We define *rescheduling* as changing the relative scheduling of syndrome qubits on a shared data qubit. This changes the timesteps at which circuit-level errors are detected on syndrome qubits.



**Figure 10.** A rescheduling change to modify the behavior of a circuit-level error corresponding to the highlighted CNOT. Since opposite type (X/Z) stabilizers are involved, a second CNOT needs to be rescheduled to preserve commutation.

For a circuit-level error originating from a CNOT between data qubit  $q_i$  and syndrome qubit  $s_j$ , we enumerate  $|S_{q_i}|$



**Figure 11.** Internal representation used in PropHunt to track relative scheduling of syndrome qubits on shared data qubits.

rescheduling changes, where  $|S_{q_i}|$  is the set of syndrome qubits flipped by the circuit-level error on  $q_i$ . For each syndrome qubit  $s_i \in S_{q_i}$ , we swap the relative ordering of  $s_i$  and  $s_j$  on qubit  $q_i$ . If  $s_i$  and  $s_j$  correspond to a pair of X and Z stabilizers, then we need to make an additional rescheduling change to preserve stabilizer commutation. We select another qubit  $q_k$  connected to both  $s_i$  and  $s_j$  and additionally swap the relative ordering of  $s_i$  and  $s_j$  on it. For many codes, such as the surface code, there are uniquely two qubits  $q_i, q_k$  shared between X and Z stabilizers, and so the selection of  $q_k$  is deterministic. Otherwise,  $q_k$  is selected at random.

Internally, PropHunt uses a multi-edge, directed graph representation (Figure 11) to simplify the tracking of rescheduling changes. Each syndrome qubit corresponds to a node in the graph, and an edge is added between pairs of nodes for each shared data qubit. The direction of edges indicate which syndrome qubit interacts with the data qubit first. This implicitly models the propagation of errors, as an out edge from a syndrome node indicates a CNOT error between the syndrome and data qubit pair can propagate to the connected syndrome node *the same round*. In edges similarly indicate a CNOT error can propagate to the connected syndrome node *the next round*.

With this representation, rescheduling changes amount to flipping the direction of edges. This graph coupled with specified syndrome qubit orderings as described in Section 5.3.1, define CNOT ordering dependencies that can then be directly converted to a CNOT schedule.

### 5.4 Pruning SM Circuit Changes

In the previous stage we produce  $O(wd_{\text{eff}})$  candidate changes to the SM circuit for each minimum weight-logical error found. Most of these changes will not improve the code's logical error rate. To prune the set of candidate changes, we implement two checks:

**Circuit Validity:** Not all candidate changes correspond to valid SM circuits, and candidate changes creating invalid SM

**Table 1.** Selected QEC Codes for Evaluation

| Code Construction | Code Parameters |
|-------------------|-----------------|
| Surface Codes     | [[9,1,3]]       |
|                   | [[25, 1, 5]]    |
|                   | [[49, 1, 7]]    |
|                   | [[81, 1, 9]]    |
| LP Codes          | [[39, 3, 3]]    |
| RQT Codes         | [[54, 11, 4]]   |
|                   | [[108, 18, 4]]  |
|                   | [[60, 2, 6]]    |

circuits are excluded. We verify SM circuit validity by checking ① stabilizer commutation is preserved and ② all CNOTs are schedulable. For ②, not schedulable CNOTs arise from cycles in the graph structure shown in Figure 11 creating a circular dependence.

**Ambiguity Removal:** Candidate changes originate from circuit-level errors,  $E'$ , involved in ambiguous errors found according to Sections 5.1 and 5.2. We can therefore check if a candidate change resolves the ambiguous errors. This involves checking  $L' \in \text{rowsp}(H')$  for updated circuit-level matrices  $H', L'$  corresponding to the original ambiguous syndrome bits now in the candidate change’s SM circuit.

We additionally need to check the updated circuit-level errors are not still a logical error. Specifically, that  $H'E' \neq 0$  or  $L'E' = 0$ .

### 5.5 Applying Verified SM Circuit Changes

After pruning candidate SM circuit changes, we will have a set of verified changes that resolve the ambiguity found in Section 5.1. We attempt to apply all verified changes to the SM circuit, however, there may be instances where multiple changes are incompatible. For example, if we find multiple verified circuit changes for a single ambiguous subgraph then applying one of them invalidates the ambiguity analysis for the others as the ambiguity in the subgraph has been removed. We therefore only apply the change corresponding to the shortest depth SM circuit in such cases. Circuit depth is a natural secondary optimization target here, as after Section 5.4 all changes resolve ambiguity and circuit depth is still relevant for minimizing decoherence errors.

## 6 Evaluation

We evaluate PropHunt on a suite of QEC codes shown in Table 1. Lifted Product (LP) and Random Quantum Tanner (RQT) codes are constructed using the qLDPC library [39]. The  $[[39, 3, 3]]$  LP code is constructed using the cyclic group  $C_3$  and the protograph in Equation 8 of [47]. The stabilizer matrices are a mix of weight 4, 5, and 6 stabilizers. The  $[[60, 2, 6]]$  RQT code is constructed from a length-2 repetition code and the cyclic group  $C_{15}$ . The stabilizer matrices contain weight 4 stabilizers. Finally the  $[[54, 11, 4]]$  and

$[[108, 18, 4]]$  RQT codes are both created using length-3 repetition codes and the Dihedral group  $D_6$  [43]. The stabilizer matrices contain weight 6 stabilizers.

### 6.1 Optimizing Benchmark Codes

We first discuss PropHunt’s ability to optimize the benchmark QEC codes in Table 1.

We select the coloration circuit described in Algorithm 1 of [56] as our baseline. This is a generally applicable SM circuit technique for CSS codes and is currently employed by popular open-source libraries for circuit-level simulations [39, 46].

All experiments start from the baseline circuit and ran for a maximum of 25 iterations with 500 samples per iteration. Additionally, ambiguous subgraph finding was parallelized over 48 Intel Xeon Silver 4116 CPU cores. Logical error rate simulations of the resulting SM circuits use a standard circuit-level model of  $d$  rounds of the SM circuit. Single qubit operations have  $\{X, Y, Z\}$  applied with probability  $\frac{p}{3}$  after each gate and two qubit operations have  $\{I, X, Y, Z\} \otimes \{I, X, Y, Z\} \setminus II$  applied with probability  $\frac{p}{15}$  after each gate. Simulations are done using Stim [16]. Decoding for surface codes was performed using PyMatching [19], and decoding for LP and RQT codes was performed using BP-LSD [20]. All logical error rates include both logical X and Z error rates.

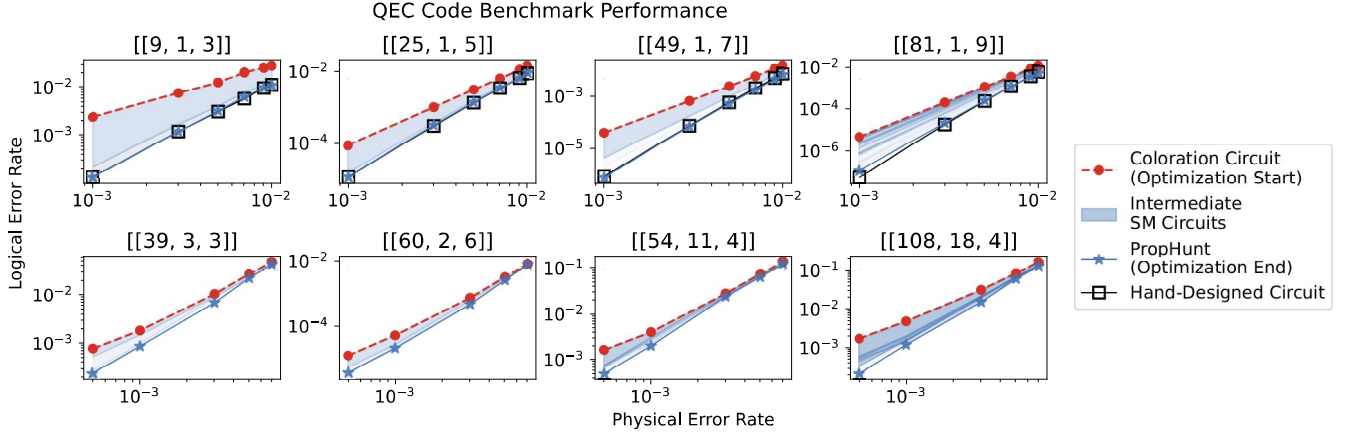
The results are shown in Figure 12. We see that for all codes, PropHunt is able to improve logical error rate compared to the coloration circuit. For the surface codes, we see PropHunt produces SM circuits that match the performance of well-known, hand-designed circuits, indicating PropHunt’s ability to successfully automate SM circuit optimization. For the LP and RQT codes where performant SM circuits are unknown, PropHunt successfully identifies and resolves problematic CNOT ordering and scheduling that causes reduced performance. Compared to the coloration circuit, this improves logical error rates by 2.5x-4x at a physical error rate of 0.1%.

In Figure 13 we also evaluate PropHunt on three different, random coloration circuits. We observe similar performance as in Figure 12, indicating robustness in PropHunt’s ability to improve SM circuit performance for the benchmark codes we evaluate.

### 6.2 Finding Min-Weight Ambiguous Errors

We now discuss the performance of PropHunt’s solution for finding ambiguous errors in an SM circuit. Since found ambiguous errors drive schedule changes in PropHunt, the performance of this stage is an optimization bottleneck.

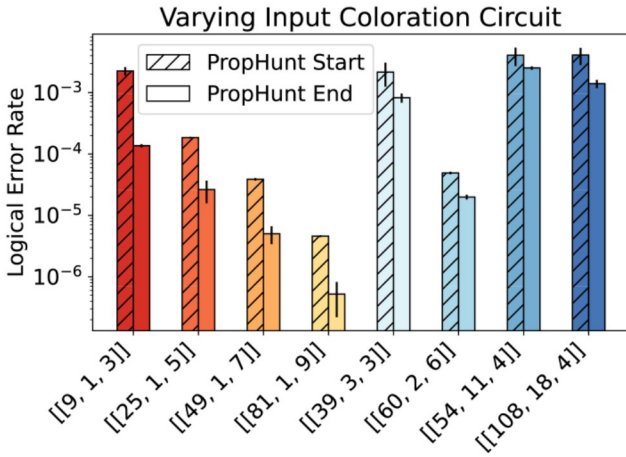
Using just a code’s stabilizer matrices, sampling is commonly used to estimate  $d$  [42]. Extending this to the circuit-level model, however, is not feasible due to the large check matrices involved as described in Section 2.7. This also makes globally solving for min-weight logical errors in the circuit-level model intractable. In Table 2 we show MaxSAT model



**Figure 12.** PropHunt’s performance on benchmark QEC codes. Logical error rates include both logical X and Z error rates. For the surface codes, performance of a popular, hand-designed circuit is included for comparison.

| Formulation | Code       | SM Circuit Distance  | Variables | Hard Clauses | Soft Clauses | Wall Clock Time |
|-------------|------------|----------------------|-----------|--------------|--------------|-----------------|
| Global      | [[39,3,3]] | $d_{\text{eff}} = 2$ | 27958     | 128709       | 7884         | 101.36 s        |
|             | [[49,1,7]] | $d_{\text{eff}} = 4$ | 45050     | 182533       | 16968        | 1 hr 55 min     |
|             | [[60,2,6]] | $d_{\text{eff}} = 4$ | 60815     | 266381       | 20880        | *               |
| Subgraph    | [[39,3,3]] | $d_{\text{eff}} = 2$ | 257       | 900          | 123          | 1.18 s          |
|             | [[49,1,7]] | $d_{\text{eff}} = 4$ | 340       | 1196         | 164          | 1.28 s          |
|             | [[60,2,6]] | $d_{\text{eff}} = 4$ | 246       | 941          | 110          | 1.39 s          |

**Table 2.** Comparing example MaxSAT model size between global solving and ambiguous subgraph solving. \* Solver timed out.



**Figure 13.** Evaluating PropHunt given three different, random coloration circuits as input. Data is for a physical error rate of 0.1%. Although variation exists in both starting and ending performance, indicated by vertical bars, PropHunt is able to consistently improve performance of the input coloration circuit.

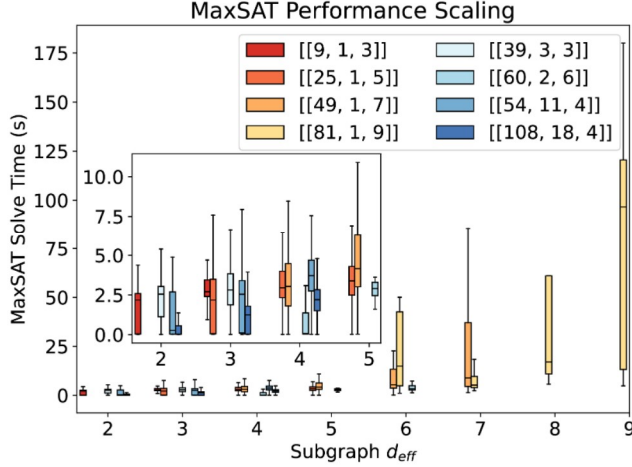
sizes based on the formulation in Section 5.2. Our ambiguous subgraph approach results in models much more tractable

than a global approach, allowing for min-weight logical errors to be found efficiently. Additionally, ambiguous subgraphs are sampled in parallel, allowing for multiple parts of a SM circuit to be optimized simultaneously.

**6.2.1 Ambiguous Subgraph MaxSAT Scaling.** In Figure 14 we plot solve times for our ambiguous subgraph formulation during PropHunt optimization. Subgraph size scales with  $d_{\text{eff}}$  and so we expect solve times to increase for larger  $d_{\text{eff}}$ . Interestingly, we observe high variability in solve times for certain codes. For example, at  $d_{\text{eff}} = 6$  the  $d = 7$  and  $d = 9$  surface codes have a notably larger range in solve times compared to the  $d = 6$  RQT code. This variability also increases at higher  $d_{\text{eff}}$ . For example, this effect is much less noticeable for the  $d = 3$  and  $d = 5$  surface codes, indicating solve time variability will likely increase with higher distance codes in the future.

### 6.3 Idle Error Sensitivity Study

PropHunt is different than prior work in that it focuses on modifying error propagation in SM circuits to reduce ambiguity, and circuit depth is not its primary optimization target. We believe this is well-motivated, as for many hardware platforms two-qubit gate layers are not bottlenecks. For example, in neutral atoms, two-qubit gate times are  $\sim 300$



**Figure 14.** Scaling analysis of our ambiguous subgraph MaxSAT formulation. Data is collected from 1000 ambiguous subgraph samples during each iteration of PropHunt optimization. Model size and solve times both scale with  $d_{\text{eff}}$  which increases during optimization, eventually saturating at  $d_{\text{eff}} = d$  for the benchmark codes we consider.

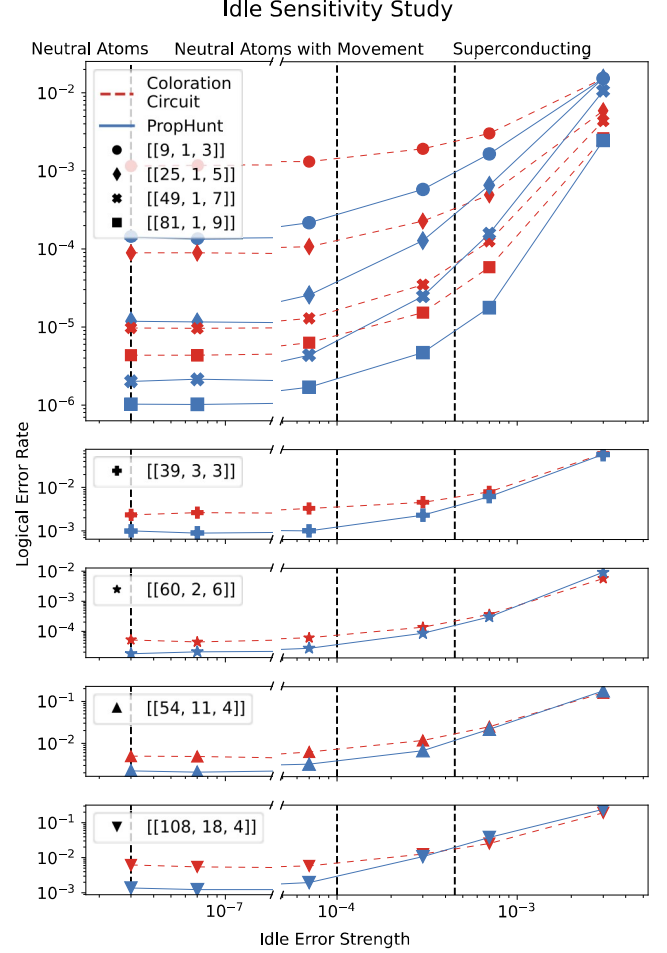
ns, compared to measurement times of  $\sim 1$  ms [5, 44]. Small increases in circuit depth can therefore be preferable if they improve the resulting logical error rate.

To understand this trade-off, we study the sensitivity of the SM circuits evaluated in Section 6.1 to idle errors between gate layers. We add idle errors to our simulations using a popular Pauli twirling approximation [55], and use a fixed gate error rate of 0.1%. We additionally include three points of interest corresponding to neutral atom systems, superconducting systems, and movement-based neutral atom systems based off of experimental results [1, 5, 32, 44]. For movement-based neutral atoms, we assume each gate layer requires  $500\mu\text{s}$  of movement, however, the exact amount of movement varies by code, with surface codes requiring less movement [6]. Movement costs could also be reduced using automated tools [31, 51, 53, 61].

The results are shown in Figure 15. Idle error strength is defined as the ratio:  $t_g/T$  where  $t_g$  is the idle time for each gate layer and  $T$  is the qubit coherence time. In general, we can see that for a large range of relevant idle error strengths, small increases in circuit depth from PropHunt are outweighed by the improvements in logical error rate. We leave further evaluation of codes for specific hardware to future work, and hope the data in Figure 15 is informative to a broad range of experimental setups.

## 7 Fine-Grained Noise Scaling for QEC-ZNE

QEC is essential for suppressing physical noise but is costly and, on near-term hardware, unlikely to achieve demanding fidelity targets on its own. A unique feature of the results in



**Figure 15.** Idle error sensitivity study for benchmark QEC codes at a gate error rate of 0.1%. Idle error strength is the ratio of two-qubit gate time to qubit coherence time. For neutral atoms with movement, we assume movement is needed between each two-qubit gate layer.

Section 6 is the intermediate logical error rates produced by PropHunt during optimization. We note that more intermediate logical error rates can easily be achieved by decreasing the samples per iteration and increasing the number of iterations, slowing down optimization convergence. We find this is a natural fit to Zero-Noise Extrapolation (ZNE) [27, 54], an error mitigation technique which intentionally increases error rates to then fit expectation values to an estimated zero-noise limit. Given this insight, we propose a ZNE-based application of PropHunt, termed *Hook-ZNE*. Since ZNE is a complementary, hardware-agnostic mitigation technique we find this promising for near-term applications of PropHunt.

### 7.1 Challenges in Current QEC–ZNE Integration

With QEC The logical error rate scales as

$$P_L(d) \approx \left(\frac{P}{P_{\text{th}}}\right)^{\lceil \frac{d}{2} \rceil} = \Lambda^{-\lceil \frac{d}{2} \rceil}, \quad \Lambda \equiv \frac{P_{\text{th}}}{P},$$

where  $d$  is the code distance,  $P$  is the physical error rate, and  $P_{\text{th}}$  is the threshold error rate. When the hardware operates below threshold ( $P < P_{\text{th}}$ ), we have  $\Lambda > 1$ , so increasing the code distance  $d$  suppresses the logical error rate  $P_L(d)$  exponentially. Utilizing this relationship among  $P_L(d)$ ,  $\Lambda$ , and  $d$ , prior works combine QEC with ZNE, yet all face challenges in effectively reducing errors.

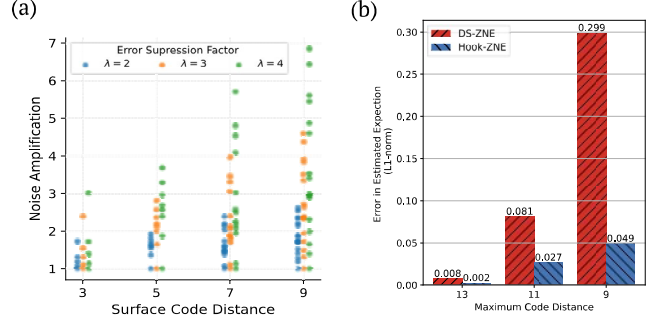
**Logical Circuit Folding** increases noise by appending inverse–forward logical blocks, effectively stretching the circuit depth. While this provides a tunable way to amplify errors, it also lengthens the execution time, which is especially costly on error-corrected hardware where operations are already slow and resource-intensive.

**Distance-Scaling ZNE (DS-ZNE)** scales the code distance to vary logical error rates [60]. If an application can run at distance  $d$ , it is also executed at  $d-2, d-4, \dots, d-2k$ . Lowering  $d$  increases noise, enabling extrapolation back to zero-noise. However, two limitations arise: ① the scaling factors are coarse, since logical error rates fall exponentially with linear increases in  $d$ , and distance values are restricted to odd integers; ② on near-term devices with small maximum distances, the variance of observables grows quickly at low  $d$ , and only a handful of  $d$  values are available, making fitting unreliable. ③ Scaling Code distance requires complete recompilation, including distance-specific mapping, routing, and Clifford + T synthesis optimizations. In theory, DS-ZNE enables higher throughput by lowering code distance and freeing qubits to run parallel copies of the computation. In practice, however, most qubits are consumed by magic state distillation, which already operates at low distance utilizing post-selection. As a result, reducing the distance of algorithmic qubits does little to free resources, making the parallelism advantage largely negligible.

**Physical-Level Noise Amplification** modifies physical gate durations or idle times at fixed  $d$  to amplify physical noise within SM circuits [68]. Unfortunately, this would require fine-grained control over physical gates used for SM. Moreover, amplifying noise without any bias at the physical level is challenging and can lead to nonlinear noise scaling and distortion of error structure, especially when the physical error rate is close to the threshold. Another challenge in scaling physical errors is increased decoding pressure due to higher physical error rates, which can make decoding more challenging and lead to unintended decoder slowdown.

### 7.2 Hook-ZNE Design and Evaluation

Hook-ZNE amplifies logical noise smoothly at fixed code distance by leveraging suboptimal SM circuits discovered during PropHunt’s optimization. This provides fine-grained,



**Figure 16.** (a) Noise Amplification with Hook-ZNE (b) Bias in expectation values for DS-ZNE vs. Hook-ZNE.

low-overhead control over the logical error rate without increasing logical circuit depth or qubit count, enabling bias–variance trade-offs that are better suited for near-term, resource-constrained fault-tolerant quantum computing. Figure 16a illustrates the range of noise amplification achievable for a surface code at fixed code distance under different suppression factors,  $\Lambda$ . For example, in Google’s surface code experiments, the reported suppression factor was  $\Lambda = 2.14$  [1].

We further evaluate **Hook-ZNE** against **DS-ZNE** by comparing the *bias*, defined as the  $L_1$  distance between the estimated and ideal expectation value of the target observable. Following the DS-ZNE setup with suppression factor  $\Lambda = 2$ , we test three distance ranges. For DS-ZNE, we use distances with odd integers. For Hook-ZNE, we round to finely spaced values of  $d$  which can be understood as fits to  $P_L(d) = \Lambda^{-\frac{d+1}{2}}$ :

DS-ZNE:  $d \in \{13, 11, 9, 7; 11, 9, 7, 5; 9, 7, 5, 3\}$

Hook-ZNE:  $d \in \{13, 12.5, 12, 11.5; 11, 10.5, 10, 9.5; 9, 8.5, 8, 7.5\}$

We use mitiq [22] to generate randomized benchmarking circuits with a two-qubit gate circuit depth of 50. Uniform gate noise is added for each gate with magnitude defined by  $P_L(d) = \Lambda^{-\frac{d+1}{2}}$ . To compare DS-ZNE versus Hook-ZNE, we use a total budget of 20,000 shots. The results are shown in Figure 16b. We can see Hook-ZNE consistently produces estimations closer to the true expectation value across all configurations. We find the key advantage of Hook-ZNE is its *finer noise scaling* at essentially fixed  $d$ , which minimizes exposure to very low distances where noise amplification and estimator variance grow rapidly. This leads to more stable fits under the same total shot budget.

## 8 Related Work

### Alternative approaches for finding SM circuits:

An alternative approach for finding performant CNOT schedules in SM circuits is to ① identify repetitive code structures to create a smaller, parameterized circuit and ② perform a brute-force search over the parameterized circuit.

This strategy has been successfully employed for bivariate bicycle codes [7] and color codes [23].

Another approach is to design SM circuits by hand, which has been successfully employed for surface codes [55] and generalized bicycle codes [28].

#### Flag fault-tolerant SM circuits:

There is related work that uses extra flag qubits to detect hook errors and create SM circuits where  $d_{\text{eff}} = d$  [8–11]. Flag-Proxy Networks [59] is an automated approach using flag qubits and proxy qubits for creating distance-preserving SM circuits that also have low connectivity requirements. In [49] the authors used SMT solvers and flag fault-tolerance to create fault-tolerant circuits for preparing  $|H\rangle$ -type magic states in the color code.

In contrast to these related works, PropHunt does not use extra ancilla qubits to detect hook errors. However, future work could explore augmenting the circuits output by PropHunt with flag fault-tolerance.

#### Related SM circuit problems:

QECC-synth [66] addressed SM circuit compilation for QEC codes that are natively incompatible with the underlying hardware architecture. This work extended and improved other prior work that used ancilla qubits for enabling QEC codes on incompatible hardware architectures [63]. Gehér et al. also introduced a tangling schedule that reduced hardware connectivity requirements for QEC codes [15].

In [3] the authors proposed an adaptive SM circuit strategy for concatenated codes where the outer codes are measured less frequently, dependent on logical soft information from the inner codes.

Automated approaches also exist for creating surface code SM circuits in the presence of defective components using high-weight superstabilizers comprised of low-weight gauge operators [13, 18, 24, 30, 50, 65].

In theory, PropHunt is complementary to these related SM circuit problems and solutions, but thoroughly exploring the connection is a step we leave for future work.

#### Alternative approaches to syndrome measurement:

Other methods of measuring syndromes exist including Shor-style [48], Steane-style [52], and Knill-style [21]. In contrast to the SM circuit method, these methods cost extra logical ancilla qubits. These logical ancilla qubits must be prepared using a state preparation circuit which is addressed in related work [38, 67].

## 9 Conclusion

In this work we addressed SM circuit optimization for CSS QEC codes. We point out imperfect performance predictors used in prior work, and introduce a novel approach based on minimizing ambiguity in SM circuits. We present a tool, PropHunt, for optimizing SM circuits and find PropHunt produces SM circuits that recover the performance of hand-designed circuits. Additionally, circuits for LP and RQT codes

produced by PropHunt have 2.5x-4x lower logical error rates than existing circuits. Finally, we show intermediate SM circuits created by PropHunt can be used for error mitigation with ZNE, which we call Hook-ZNE. Our evaluations show Hook-ZNE reduces error by 3x-6x compared to DS-ZNE.

## Acknowledgments

We would like to thank the anonymous reviewers for their helpful feedback and suggestions.

This work is funded in part by the STAQ project under award NSF Phy-232580; in part by the US Department of Energy Office of Advanced Scientific Computing Research, Accelerated Research for Quantum Computing Program; in part by the NSF Quantum Leap Challenge Institute for Hybrid Quantum Architectures and Networks (NSF Award 2016136); in part by the NSF National Quantum Virtual Laboratory program; in part based upon work supported by the U.S. Department of Energy, Office of Science, National Quantum Information Science Research Centers; in part by the Army Research Office under Grant Number W911NF-23-1-0077; in part by the U.S. Department of Energy, Office of Science, National Quantum Information Science Research Centers, Co-design Center for Quantum Advantage (C2QA) under contract number DE-SC0012704; and in part by the U.S. National Science Foundation (NSF) under Award Abstract number 2212232.

The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein. FTC is the Chief Scientist for Quantum Software at Infleqtion.

## Data Availability

The code and data for this work is open-source and available at [github.com/jviszlai/PropHunt](https://github.com/jviszlai/PropHunt).

## References

- [1] 2025. Quantum error correction below the surface code threshold. *Nature* 638, 8052 (2025), 920–926.
- [2] Jeremias Berg, Emir Demirović, and Peter J Stuckey. 2019. Core-booster linear search for incomplete MaxSAT. In *International conference on integration of constraint programming, artificial intelligence, and operations research*. Springer, 39–56.
- [3] Noah Berthussen, Shi Jie Samuel Tan, Eric Huang, and Daniel Gottesman. 2025. Adaptive Syndrome Extraction. *PRX Quantum* 6, 3 (July 2025). doi:10.1103/ps3r-wf84
- [4] Michael E Beverland, Aleksander Kubica, and Krysta M Svore. 2021. Cost of universality: A comparative study of the overhead of state distillation and code switching with color codes. *PRX Quantum* 2, 2 (2021), 020341.
- [5] Dolev Bluvstein, Simon J Evered, Alexandra A Geim, Sophie H Li, Hengyun Zhou, Tom Manovitz, Sepehr Ebadi, Madelyn Cain, Marcin Kalinowski, Dominik Hangleiter, et al. 2024. Logical quantum processor based on reconfigurable atom arrays. *Nature* 626, 7997 (2024),

58–65.

- [6] Dolev Bluvstein, Harry Levine, Giulia Semeghini, Tout T Wang, Sepehr Ebadi, Marcin Kalinowski, Alexander Keesling, Nishad Maskara, Hannes Pichler, Markus Greiner, et al. 2022. A quantum processor based on coherent transport of entangled atom arrays. *Nature* 604, 7906 (2022), 451–456.
- [7] Sergey Bravyi, Andrew W Cross, Jay M Gambetta, Dmitri Maslov, Patrick Rall, and Theodore J Yoder. 2024. High-threshold and low-overhead fault-tolerant quantum memory. *Nature* 627, 8005 (2024), 778–782.
- [8] Christopher Chamberland and Michael E. Beverland. 2018. Flag fault-tolerant error correction with arbitrary distance codes. *Quantum* 2 (Feb. 2018), 53. doi:10.22331/q-2018-02-08-53
- [9] Christopher Chamberland, Guanyu Zhu, Theodore J. Yoder, Jared B. Hertzberg, and Andrew W. Cross. 2020. Topological and Subsystem Codes on Low-Degree Graphs with Flag Qubits. *Physical Review X* 10, 1 (Jan. 2020). doi:10.1103/physrevx.10.011022
- [10] Rui Chao and Ben W. Reichardt. 2018. Fault-tolerant quantum computation with few qubits. *npj Quantum Information* 4, 1 (Sept. 2018). doi:10.1038/s41534-018-0085-z
- [11] Rui Chao and Ben W Reichardt. 2020. Flag fault-tolerant error correction for any stabilizer code. *PRX Quantum* 1, 1 (2020), 010302.
- [12] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340.
- [13] Dripto M Debroy, Matt McEwen, Craig Gidney, Noah Shutt, and Adam Zalcman. 2025. Luci in the surface code with dropouts. *Quantum* 9 (2025), 1936.
- [14] Yongshan Ding and Frederic T Chong. 2020. Quantum computer systems: Research for noisy intermediate-scale quantum computers. *Synthesis lectures on computer architecture* 15, 2 (2020), 1–227.
- [15] György P. Gehér, Ophelia Crawford, and Earl T. Campbell. 2024. Tangling Schedules Eases Hardware Connectivity Requirements for Quantum Error Correction. *PRX Quantum* 5, 1 (March 2024). doi:10.1103/prxquantum.5.010348
- [16] Craig Gidney. 2021. Stim: a fast stabilizer circuit simulator. *Quantum* 5 (July 2021), 497. doi:10.22331/q-2021-07-06-497
- [17] Matthew B Hastings, Jeongwan Haah, and Ryan O’Donnell. 2021. Fiber bundle codes: breaking the  $n^{1/2} \text{polylog}(n)$  barrier for quantum ldpc codes. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*. 1276–1288.
- [18] Oscar Higgott, Benjamin Anker, Matt McEwen, and Dripto M Debroy. 2025. Handling fabrication defects in hex-grid surface codes. *arXiv preprint arXiv:2508.08116* (2025).
- [19] Oscar Higgott and Craig Gidney. 2025. Sparse Blossom: correcting a million errors per core second with minimum-weight matching. *Quantum* 9 (Jan. 2025), 1600. doi:10.22331/q-2025-01-20-1600
- [20] Timo Hillmann, Lucas Berent, Armanda O Quintavalle, Jens Eisert, Robert Wille, and Joschka Roffe. 2024. Localized statistics decoding: A parallel decoding algorithm for quantum low-density parity-check codes. *arXiv preprint arXiv:2406.18655* (2024).
- [21] Emanuel Knill. 2005. Quantum computing with realistically noisy devices. *Nature* 434, 7029 (2005), 39–44.
- [22] Ryan LaRose, Andrea Mari, Sarah Kaiser, Peter J Karalekas, Andre A Alves, Piotr Czarnik, Mohamed El Mandouh, Max H Gordon, Yousef Hindy, Aaron Robertson, et al. 2022. Mitiq: A software package for error mitigation on noisy quantum computers. *Quantum* 6 (2022), 774.
- [23] Seok-Hyung Lee, Andrew Li, and Stephen D Bartlett. 2025. Color code decoder with improved scaling for correcting circuit-level noise. *Quantum* 9 (2025), 1609.
- [24] Catherine Leroux, Sophia F Lin, Przemyslaw Bienias, Krishanu R Sankar, Asmae Benhemou, Aleksander Kubica, and Joseph K Iverson. 2024. Snakes and Ladders: Adapting the surface code to defects. *arXiv preprint arXiv:2412.11504* (2024).
- [25] Anthony Leverrier and Gilles Zémor. 2022. Quantum tanner codes. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 872–883.
- [26] Gushu Li, Yufei Ding, and Yuan Xie. 2019. Tackling the qubit mapping problem for NISQ-era quantum devices. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 1001–1014.
- [27] Ying Li and Simon C Benjamin. 2017. Efficient variational quantum simulator incorporating active error minimization. *Physical Review X* 7, 2 (2017), 021050.
- [28] Hsiang-Ku Lin, Xingrui Liu, Pak Kau Lim, and Leonid P Pryadko. 2025. Single-shot and two-shot decoding with generalized bicycle codes. *arXiv preprint arXiv:2502.19406* (2025).
- [29] Hsiang-Ku Lin and Leonid P Pryadko. 2024. Quantum two-block group algebra codes. *Physical Review A* 109, 2 (2024), 022407.
- [30] Sophia Fuhui Lin, Joshua Vizslai, Kaitlin N Smith, Gokul Subramanian Ravi, Charles Yuan, Frederic T Chong, and Benjamin J Brown. 2024. Codeign of quantum error-correcting codes and modular chiplets in the presence of defects. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 216–231.
- [31] Wan-Hsuan Lin, Daniel Bochen Tan, and Jason Cong. 2025. Reuse-aware compilation for zoned quantum architectures based on neutral atoms. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 127–142.
- [32] Shuo Ma, Alex P Burgers, Genyue Liu, Jack Wilson, Bichen Zhang, and Jeff D Thompson. 2022. Universal gate operations on nuclear spin qubits in an optical tweezer array of Yb 171 atoms. *Physical Review X* 12, 2 (2022), 021028.
- [33] Alexander J Malcolm, Andrew N Glaudell, Patricio Fuentes, Daryus Chandra, Alexis Schotte, Colby DeLisle, Rafael Haenel, Amir Ebrahimi, Joschka Roffe, Armanda O Quintavalle, et al. 2025. Computing efficiently in QLDPC codes. *arXiv preprint arXiv:2502.07150* (2025).
- [34] Argyris Giannisis Manes and Jahan Claes. 2025. Distance-preserving stabilizer measurements in hypergraph product codes. *Quantum* 9 (2025), 1618.
- [35] Abtin Molavi, Amanda Xu, Martin Diges, Lauren Pick, Swamit Tannu, and Aws Albarghouthi. 2022. Qubit mapping and routing via MaxSAT. In *2022 55th IEEE/ACM international symposium on Microarchitecture (MICRO)*. IEEE, 1078–1091.
- [36] Michael A Nielsen and Isaac L Chuang. 2001. Quantum computation and quantum information. *Phys. Today* 54, 2 (2001), 60.
- [37] Laura Pecorari, Sven Jandura, Gavin K Brennen, and Guido Pupillo. 2025. High-rate quantum LDPC codes for long-range-connected neutral atom registers. *Nature Communications* 16, 1 (2025), 1111.
- [38] Tom Peham, Ludwig Schmid, Lucas Berent, Markus Müller, and Robert Wille. 2025. Automated Synthesis of Fault-Tolerant State Preparation Circuits for Quantum Error-Correction Codes. *PRX Quantum* 6, 2 (2025), 020330.
- [39] Michael A. Perlin. 2023. qLDPC. <https://github.com/qLDPCOrg/qLDPC>.
- [40] C Poole, TM Graham, MA Perlin, M Otten, and M Saffman. 2024. Architecture for fast implementation of qLDPC codes with optimized Rydberg gates. *arXiv preprint arXiv:2404.18809* (2024).
- [41] Steven Prestwich. 2021. CNF encodings. *Handbook of satisfiability* (2021), 75–100.
- [42] Leonid P Pryadko, Vadim A Shabashov, and Valerii K Kozin. 2023. QDistRnd: A GAP package for computing the distance of quantum error-correcting codes. *arXiv preprint arXiv:2308.15140* (2023).
- [43] Rebecca Katharina Radebold, Stephen D Bartlett, and Andrew C Doherty. 2025. Explicit Instances of Quantum Tanner Codes. *arXiv preprint arXiv:2508.05095* (2025).
- [44] AG Radnaev, WC Chung, DC Cole, D Mason, TG Ballance, MJ Bedalov, DA Belknap, MR Berman, M Blakely, IL Bloomfield, et al. 2024. A

- universal neutral-atom quantum computer with individual optical addressing and non-destructive readout. *arXiv preprint arXiv:2408.08288* (2024).
- [45] Eleanor G Rieffel and Wolfgang H Polak. 2011. *Quantum computing: A gentle introduction*. MIT Press.
- [46] Joschka Roffe. 2022. *LDPC: Python tools for low density parity check codes*. <https://pypi.org/project/ldpc/>
- [47] Joschka Roffe, Lawrence Z Cohen, Armanda O Quintavalle, Daryus Chandra, and Earl T Campbell. 2023. Bias-tailored quantum LDPC codes. *Quantum* 7 (2023), 1005.
- [48] Peter W Shor. 1996. Fault-tolerant quantum computation. In *Proceedings of 37th conference on foundations of computer science*. IEEE, 56–65.
- [49] Noah Shutty and Christopher Chamberland. 2022. Decoding Merged Color-Surface Codes and Finding Fault-Tolerant Clifford Circuits Using Solvers for Satisfiability Modulo Theories. *Physical Review Applied* 18, 1 (July 2022). doi:10.1103/physrevapplied.18.014072
- [50] Adam Siegel, Armands Strikis, Thomas Flatters, and Simon Benjamin. 2023. Adaptive surface code for quantum error correction in the presence of temporary or permanent defects. *Quantum* 7 (2023), 1065.
- [51] Yannick Stade, Ludwig Schmid, Lukas Burgholzer, and Robert Wille. 2024. An abstract model and efficient routing for logical entangling gates on zoned neutral atom architectures. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 1. IEEE, 784–795.
- [52] Andrew M Steane. 1997. Active stabilization, quantum computation, and quantum state synthesis. *Physical Review Letters* 78, 11 (1997), 2252.
- [53] Daniel Bochen Tan, Wan-Hsuan Lin, and Jason Cong. 2025. Compilation for dynamically field-programmable qubit arrays with efficient and provably near-optimal scheduling. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*. 921–929.
- [54] Kristan Temme, Sergey Bravyi, and Jay M Gambetta. 2016. Error mitigation for short-depth quantum circuits. *arXiv preprint arXiv:1612.02058* (2016).
- [55] Yu Tomita and Krysta M Svore. 2014. Low-distance surface codes under realistic quantum noise. *Physical Review A* 90, 6 (2014), 062320.
- [56] Maxime A Tremblay, Nicolas Delfosse, and Michael E Beverland. 2022. Constant-overhead quantum error correction with thin planar connectivity. *Physical Review Letters* 129, 5 (2022), 050504.
- [57] Grigori S Tseitin. 1983. On the complexity of derivation in propositional calculus. In *Automation of reasoning: 2: Classical papers on computational logic 1967–1970*. Springer, 466–483.
- [58] Joshua Vizslai, Willers Yang, Sophia Fuhui Lin, Junyu Liu, Natalia Nottingham, Jonathan M Baker, and Frederic T Chong. 2023. Matching generalized-bicycle codes to neutral atoms for low-overhead fault-tolerance. *arXiv preprint arXiv:2311.16980* (2023).
- [59] Suhas Vittal, Ali Javadi-Abhari, Andrew W Cross, Lev S Bishop, and Moinuddin Qureshi. 2024. Flag-Proxy Networks: Overcoming the Architectural, Scheduling and Decoding Obstacles of Quantum LDPC Codes. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 718–734.
- [60] Misty A Wahl, Andrea Mari, Nathan Shammah, William J Zeng, and Gokul Subramanian Ravi. 2023. Zero noise extrapolation on logical qubits by scaling the error correction code distance. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 1. IEEE, 888–897.
- [61] Hanrui Wang, Pengyu Liu, Daniel Bochen Tan, Yilian Liu, Jiaqi Gu, David Z Pan, Jason Cong, Umut A Acar, and Song Han. 2024. Atomique: A quantum compiler for reconfigurable neutral atom arrays. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 293–309.
- [62] Stasiu Wolanski and Ben Barber. 2024. Ambiguity Clustering: an accurate and efficient decoder for qLDPC codes. *arXiv preprint arXiv:2406.14527* (2024).
- [63] Anbang Wu, Gushu Li, Hezi Zhang, Gian Giacomo Guerreschi, Yufei Ding, and Yuan Xie. 2022. A synthesis framework for stitching surface code with superconducting quantum devices. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA '22)*. ACM, 337–350. doi:10.1145/3470496.3527381
- [64] Qian Xu, J Pablo Bonilla Ataides, Christopher A Pattison, Nithin Raveendran, Dolev Bluvstein, Jonathan Wurtz, Bane Vasić, Mikhail D Lukin, Liang Jiang, and Hengyun Zhou. 2024. Constant-overhead fault-tolerant quantum computation with reconfigurable atom arrays. *Nature Physics* 20, 7 (2024), 1084–1090.
- [65] Keyi Yin, Xiang Fang, Travis S Humble, Ang Li, Yunong Shi, and Yufei Ding. 2024. Surf-Deformer: Mitigating dynamic defects on surface code via adaptive deformation. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 750–764.
- [66] Keyi Yin, Hezi Zhang, Xiang Fang, Yunong Shi, Travis S. Humble, Ang Li, and Yufei Ding. 2025. QECC-Synth: A Layout Synthesizer for Quantum Error Correction Codes on Sparse Architectures. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*. ACM, 876–890. doi:10.1145/3669940.3707236
- [67] Remmy Zen, Jan Olle, Luis Colmenarez, Matteo Puviani, Markus Müller, and Florian Marquardt. 2025. Quantum circuit discovery for fault-tolerant logical state preparation with reinforcement learning. *Physical Review X* 15, 4 (2025), 041012.
- [68] Aosai Zhang, Haipeng Xie, Yu Gao, Jia-Nan Yang, Zehang Bao, Zitian Zhu, Jiachen Chen, Ning Wang, Chuanyu Zhang, Jiarun Zhong, et al. 2025. Demonstrating quantum error mitigation on logical qubits. *arXiv preprint arXiv:2501.09079* (2025).
- [69] Alwin Zulehner, Alexandru Paler, and Robert Wille. 2018. An efficient methodology for mapping quantum circuits to the IBM QX architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 7 (2018), 1226–1236.

## A Artifact Appendix

### A.1 Abstract

The artifact includes the source code, data, and scripts necessary to reproduce the results in this paper. Additionally, it contains our introduced tool, PropHunt, for optimizing syndrome measurement circuits of CSS codes.

### A.2 Artifact check-list (meta-information)

All details, including software and hardware requirements, are summarized below:

- **License:** MIT License
- **Software requirements:** Docker, Python
- **Hardware:** Multi-core CPU
- **How much disk space required (approximately)?:** 2GB
- **How much time is needed to prepare workflow (approximately)?:** 5-10 minutes to build Docker image
- **How much time is needed to complete experiments (approximately)?:** 8-12 hours depending on number of CPU cores
- **Publicly available?:** Yes
- **Archived (provide DOI)?:** 10.5281/zenodo.17945386

### A.3 Description

**A.3.1 How to access.** The most up-to-date version of the code can be accessed by cloning the repository from GitHub:

```
git clone https://github.com/jvizslai/PropHunt.git
```

Alternatively the archived artifact version can be accessed by downloading and unzipping the Zenodo archive.

#### A.4 Installation

To simplify the building and integration of the Loandra MaxSAT solver into PropHunt, we’ve provided a Docker image for data generation. We’ve also created a Makefile to simplify evaluation.

The Docker image can be created most easily by running:

```
make docker
```

from the main directory.

Plotting uses Python. numpy and matplotlib are required.

#### A.5 Experiment workflow

Scripts for running experiments are provided in the scripts directory. Running all experiments using the Docker image can be done by:

```
make data
```

from the main directory. Produced data is mounted to the local scripts/data directory to be used outside the Docker container.

scripts/plot\_data.py will generate figures for the manuscript in the scripts/figures directory. This can also be done by:

```
make plots
```

from the main directory.

#### A.6 Evaluation and expected results

Simulations should generate results for:

- Motivational data on distance and  $d_{\text{eff}}$  as performance predictors
- Hand-optimized surface code SM circuit performance
- MaxSAT solver timing results for codes in Table 2
- PropHunt results for benchmark codes compared to coloration circuit
- Sensitivity study to idle errors
- Comparison between DS-ZNE and Hook-ZNE

Plotting should generate plots or subplots for figures 1, 12, 15, and 16.

#### A.7 Experiment customization

NUM\_CORES in run\_experiments.sh should be changed to the number of cores on the system.

Additionally, the number of random samples per iteration, the number of iterations, and the number of cores used in PropHunt can be customized per benchmark in run\_experiments.sh. For example:

```
python prophunt_experiment.py surface 3 100 5 48
```

specifies the  $d = 3$  surface code benchmark, with 100 samples per iteration, for 5 iterations, run on 48 cores.